P1467R1
Extended floating-point types

Published Proposal, 2019-06-17

This version:
https://wg21.link/P1467

Authors:
Michał Dominiak (NVIDIA)
David Olsen (NVIDIA)

Audience:
SG6, EWG, LEWG

Toggle Diffs:
☐ Hide deleted text

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

§ 1. Abstract

This proposal is the less evolutionary part of [P1468], that attempts to ultimately provide the same functionality of [P0192] in a way that we expect to be more acceptable to the committee than the previous attempt.

This paper introduces the notion of *extended floating-point types*, modeled after extended integer types. To accomodate them, this paper also attempts to rewrite the current rules for floating-point types, to enable well-defined interactions between all the floating-point types. The end goal of this paper, together with [P1468], is to have a language to enable `<cstdint>`-like aliases for implementation specific floating point types, that can model more binary layouts than just a single fundamental type (the previously proposed `short float`) can provide for.

It also attempts to rewrite existing specification for both the core language and the library to not spell out all standard floating-point types every time.

§ 2. Revision history

§ 2.1. R0 -> R1 (pre-Cologne)

Applied SG6 guidance:

1. Make the floating-point conversion rank not ordered between types with overlapping (but not subsetted) ranges of finite values. This makes the ranking a partial order.
2. Narrowing conversions are now based on floating-point conversion rank instead of ranges of finite values, which preserves the current narrowing conversions relations between standard floating-point types; it also interacts favorably with the rank being a partial ordering.
3. Operations that deal with floating-point types whose conversion ranks are unordered are now ill-formed.

4. The relevant parts of the guidance have been applied to the library wording section as well.

Afterwards, applied suggestions from EWGI (this modifies some of the points above):

1. Apply the suggestion to make types where one has a wider range of finite values, but a lower precision than the other, unordered in their conversion rank, and therefore make operations that mix them ill-formed. The motivating example was IEEE-754 `binary16` and `bfloat16`; see Floating-point conversion rank for more details. This change also caused this paper to drop the term "range of finite values", since the modified semantics are better expressed in terms of sets of values of the types.
2. Add a change to narrowing conversions, to only allow exact conversions to happen (see the last paragraph of Narrowing conversions).
3. Explicitly list parts of the language that are not changed by this paper; provide a more detailed analysis of the standard library impact.

§ 3. Motivation

The motivation for the general effort of this paper is the same as for [P0192]. The entire motivation is not repeated here, but the quick summary is that 16-bit floating-point support is becoming more widely available, both in hardware (ARM CPUs and NVIDIA GPUs) and software (OpenGL, CUDA, and LLVM IR). Providing a standard way for implementations to support 16-bit floating-point types will result in better code, more portable code, and wider use of those types.

The motivation for taking the currently proposed approach comes from the result of discussion on the previous paper. Several people raised concerns about introducing just a single new fundamental type without a well-defined layout; those same people were not satisfied with the option of having a dual ABI for that type when both IEEE-754 `binary16` and `bfloat` are needed in the same application.

This paper legitimizes implementation-specific floating-point types, which makes standardizing an existing practice an additional motivation for solving the need in the way described below.

§ 4. Proposed approach

In a nutshell:

1. Introduce the notion of *extended floating-point types*.
2. Redefine usual arithmetic conversions in terms of *floating-point conversion rank*, closely modeled after the integer equivalent.

3. Redefine narrowing conversions for floating-point types, to be defined in terms the floating-point conversion rank.
4. Rewrite parts of the standard library spec as appropriate to use the new floating-point terms and rules.

§ 4.1. Aspects of the core language we aren't proposing to change

Implementations currently define whether or not each type supports infinity and NaN. This paper does not change that, still leaving those decisions up to the implementation.

Implementations currently define the radix of the exponent of each floating-point type. This paper does not change that, still leaving those decisions up to the implementation.

§ 4.2. Finer design details

Here's a list of the details of the design of this paper that we think are important; we'd like guidance on whether the committee likes the decision we've made, or if a change to them is requested; please consider them as proposed polls to determine that.

§ 4.2.1. Floating-point conversion rank

The standard has always defined *conversion rank* for integral types. This paper extends the notion of conversion rank to floating-point types.

R0 of this paper used the *range of finite values* to determine conversion rank, the rationale being that conversions from types with narrower ranges (excluding infinities) to types with wider ranges should be preferred, even if there is some loss of precision. The paper provided a total order for floating-point types, with an implementation-defined order for types whose ranges do not have a proper subset relationship.

In Kona, SG6 recommended leaving types unordered when neither type's range of finite values is a subset of the other's. Any operation that mixes unordered types would be ill-formed. This would leave the door open for inventing semantics for such operations in the future.

Later in Kona, during EWGI discussions, a concern was raised about conversions from a lower-ranked type to a higher-ranked type (as defined in R0) that would cause a loss in precision. The concern is that it is not clear how to handle types where there is a proper subset between their *ranges* of finite values, but not a matching subset relationship between the *sets* of finite values in that range. The specific case

where this would happen today is between IEEE-754 `binary16` and `bfloat16`. `bfloat16`, with an 8-bit exponent, has a much greater range that `binary16`, with a 5-bit exponent, making conversions from `binary16` to `bfloat16` implicit according to the rules in R0. This implicit conversion was worrisome because it results in a significant loss of precision, going from an 11-bit mantissa to an 8-bit mantissa. It has been suggested that the paper be revised so that implicit conversions do not result in a loss of precision.

These issues are resolved in this revision of the paper by changing the definition of floating-point conversion rank to use the *set of values* rather than the range of finite values. If two types have sets of values where neither set is a subset of the other, then the types are unordered by conversion rank and neither type will be converted into the other during the usual arithmetic conversions. Mixing unordered types in an operation is ill-formed. Conversions between unordered types is still possible with an explicit `static_cast`, but there won't be any implicit conversions in either direction.

(The use of *set of values* rather than *set of finite value* is intentional. When using ranges, infinities get in the way because all types that support infinity have the same range; it was the finite ranges that are more interesting. But when using sets of values rather than ranges, infinity is just another value. So the word *finite* is not needed any longer when defining conversion rank.)

§ 4.2.2. Narrowing conversions

Revision R0 of this paper proposed a definition of *narrowing conversion* that was not based on conversion rank and that allowed a conversion from `float` to `double` to be a non-narrowing conversion if `float` and `double` had the same representation. SG6 in Kona didn't like that idea, and that approach has been abandoned in favor of what was originally presented as an alternative.

This paper now defines a narrowing conversion as a conversion from a type with higher floating-point conversion rank to one with a lower conversion rank, or as a conversion between two types that are unordered by conversion rank. This preserves the existing behavior for standard floating-point types while extending that behavior to extended floating-point types in a consistent way.

A topic of discussion for the committee is what to do when attempting a narrowing conversion where the source is a constant expression. The paper currently leaves unchanged the wording in [dcl.init.list] p7.2: "except where the source is a constant expression and the actual value after conversion is within the range of values that can be represented (even if it cannot be represented exactly)." This behavior cannot be changed for standard floating-point types, but it might be reasonable to mandate that the value must be represented exactly when converting to an extended floating-point type.

§ 4.3. Standard library impact

Specification changes in the standard library will be required in:

1. `<cmath>`: because operations on smaller floating-point types is the primary motivation for this feature.
2. `<complex>`: for the same reason.
3. `<charconv>`: because there should be some support for I/O of extended floating-point types, and because the existing specification already supports extended integer types.
4. `<format>`: once [P0645] is adopted.

There are parts of the standard library that mention floating-point types collectively rather than listing `float`, `double`, and `long double` explicitly. The implementations of those things might need to change to handle extended floating-point types, but no specification changes are necessary.

1. `std::numeric_limits`,
2. `std::is_floating_point`,
3. `std::midpoint`.

Intentional omissions in standard library support:

1. The header `<cfloat>` provides a set of C-style macros informing of the properties of `float`, `double` and `long double`. Since this paper does not introduce any new standard floating-point types, no changes to this header are proposed.
2. No streaming operations are supported for floating-point types. Properly (that is, without losing precision and/or range of values) supporting streaming operations would require support in `num_get` and `num_put`; those classes use virtual functions, so adding an extended floating-point type would necessitate an ABI break for the standard library.
3. `std::stof` and family, because no new standard floating-point types are introduced.
4. `std::to_string` family. They are defined in terms of `snprinf`; we do not propose changing the legacy C formatting facilities for this feature.
5. [rand.req], for consistency with extended integer types.
6. The header `<atomic>` provides specializations for `std::atomic` and `std::atomic_ref` for `float`, `double` and `long double`. This list will be expanded, similarly to how it currently includes all other types necessary for aliases in `<cstdint>`, in the companion paper of this paper, [P1468], which proposes a close analogue for `<cstdint>`.

§ 5. Proposed wording

The wording changes in this paper are relative to N4810.

§ 5.1. Core language

Modify Fundamental types [**basic.fundamental**] paragraph 12:

> There are three *standard floating-point types*: `float`, `double`, and `long double`. The type `double` provides at least as much precision as `float`, and the type `long double` provides at least as much precision as `double`. The set of values of the type `float` is a subset of the set of values of the type `double`; the set of values of the type `double` is a subset of the set of values of the type `long double`. The value representation of standard floating-point types is implementation-defined. There may also be implementation-defined *extended floating-point types*. The standard and extended floating-point types are collectively called *floating-point types*. [...]

Rename ~~Integer conversion rank~~ [**conv.rank**] to Conversion ranks and insert a new paragraph at the end:

2. Every floating-point type has an *floating-point conversion rank* defined as follows:

   - (2.1) The rank of a floating point type `T` shall be greater than the rank of any floating-point type whose set of values is a proper subset of the set of values of `T`.
   - (2.2) The rank of `long double` shall be greater than the rank of `double`, which shall be greater than the rank of `float`.
   - (2.3) The rank of any standard floating-point type shall be greater than the rank of any extended floating-point type with the same set of values.
   - (2.4) The rank of any extended floating-point type relative to another extended floating-point type with the same set of values is implementation-defined, but still subject to the other rules for determining the floating-point conversion rank.
   - (2.5) For all floating-point types `T1`, `T2` and `T3`, if `T1` has greater rank than `T2` and `T2` has greater rank than `T3`, then `T1` shall have greater rank than `T3`.

   [*Note*: The conversion ranks of extended floating-point types `T1` and `T2` will be unordered if the set of values of `T1` is neither a subset nor a superset of the set of values of `T2`. This can happen when one type has both a larger range and a lower precision than the other. -- *end note*] [*Note*: The floating-point conversion rank is used in the definition of the usual arithmetic conversions ([expr.arith.conv]). -- *end note*]

Modify Floating-point promotion [**conv.fpprom**] paragraph 1:

1. A prvalue of a floating-point type ~~`float`~~ whose floating-point conversion rank ([conv.rank]) is less than the rank of `double` can be converted to a prvalue of type `double`. The value is unchanged.

Modify Usual arithmetic conversions [**expr.arith.conv**] paragraph 1:

- (1.1) If either operand is of scoped enumeration type, no conversions are performed; if the other operand does not have the same type, the expression is ill-formed.
- ~~(1.2) If either operand is of type long double, the other shall be converted to long double.~~
- ~~(1.3) Otherwise, if either operand is double, the other shall be converted to double.~~
- ~~(1.4) Otherwise, if either operand is float, the other shall be converted to float.~~
- (1.2) Otherwise, if either operand has a floating-point type, the following rules shall be applied:
  - (1.2.1) If both operands have the same type, no further conversion is needed.
  - (1.2.2) Otherwise, if one of the operands has a type that is not a floating-point type, that operand shall be converted to the type of the operand with floating-point type.
  - (1.2.3) Otherwise, if the floating-point conversion ranks ([conv.rank]) of the types of the operands are ordered, then the operand with the type of the lower floating-point conversion rank shall be converted to the type of the other operand.
  - (1.2.4) Otherwise, the expression is ill-formed.
- (~~1.5~~ 1.3) Otherwise, the integral promotions [...]

Modify the definition of narrowing conversions in List-initialization [**dcl.init.list**] paragraph 7 item 2:

- (7.2) ~~from `long double` to `double` or `float`, or from `double` to `float`~~ from a floating-point type `T` to another floating-point type whose floating-point conversion rank is not greater than that of `T` , except where the source is a constant expression and the actual value after conversion is within the range of values that can be represented (even if it cannot be represented exactly), or

§ 5.2. Library

Modify Header `<charconv>` synopsis [**charconv.syn**]:

[...]

```
  to_chars_result to_chars(char* first, char* last, *see below* value, i
```

```
- to_chars_result to_chars(char* first, char* last, float value);
- to_chars_result to_chars(char* first, char* last, double value);
- to_chars_result to_chars(char* first, char* last, long double value);

- to_chars_result to_chars(char* first, char* last, float value, chars_f
- to_chars_result to_chars(char* first, char* last, double value, chars_
- to_chars_result to_chars(char* first, char* last, long double value, c

- to_chars_result to_chars(char* first, char* last, float value,
-                          chars_format fmt, int precision);
- to_chars_result to_chars(char* first, char* last, double value,
-                          chars_format fmt, int precision);
- to_chars_result to_chars(char* first, char* last, long double value,
-                         chars_format fmt, int precision);
```

```
+ to_chars_result to_chars(char* first, char* last, *see below* value);
+ to_chars_result to_chars(char* first, char* last, *see below* value, c
+ to_chars_result to_chars(char* first, char* last, *see below* value,
+                          chars_format fmt, int precision);
```

[...]

```
  from_chars_result from_chars(const char* first, const char* last,
                               see below& value, int base = 10);
```

```
- from_chars_result from_chars(const char* first, const char* last, floa
-                              chars_format fmt = chars_format::general)
- from_chars_result from_chars(const char* first, const char* last, doub
-                              chars_format fmt = chars_format::general)
- from_chars_result from_chars(const char* first, const char* last, long
-                              chars_format fmt = chars_format::general)
```

> ```
>   from_chars_result from_chars(const char* first, const char* last, *see
>                                chars_format fmt = chars_format::general)
> ```
>
> [...]

Modify Primitive numeric output conversion [**charconv.to.chars**]:

[...]

~~`to_chars_result to_chars(char* first, char* last, float value);`~~
~~`to_chars_result to_chars(char* first, char* last, double value);`~~
~~`to_chars_result to_chars(char* first, char* last, long double value);`~~

`to_chars_result to_chars(char* first, char* last, *see below* value);`

8. *Effects*: `value` is converted to a string in the style of `printf` in the `"C"` locale. The conversion specifier is `f` or `e`, chosen according to the requirement for a shortest representation (see above); a tie is resolved in favor of `f`.
9. *Throws*: Nothing.
10. *Remarks*: The implementation shall provide overloads for all floating-point types as the type of parameter `value`.

~~`to_chars_result to_chars(char* first, char* last, float value, chars_f`~~
~~`to_chars_result to_chars(char* first, char* last, double value, chars_`~~
~~`to_chars_result to_chars(char* first, char* last, long double value, c`~~

`to_chars_result to_chars(char* first, char* last, *see below* value, c`

10. *Requires*: `fmt` has the value of one of the enumerators of `chars_format`.
11. *Effects*: `value` is converted to a string in the style of `printf` in the `"C"` locale.
12. *Throws*: Nothing.
13. *Remarks*: The implementation shall provide overloads for all floating-point types as the type of parameter `value`.

~~`to_chars_result to_chars(char* first, char* last, float value,`~~
~~`chars_format fmt, int precision);`~~
~~`to_chars_result to_chars(char* first, char* last, double value,`~~
~~`chars_format fmt, int precision);`~~
~~`to_chars_result to_chars(char* first, char* last, long double value,`~~
~~`chars_format fmt, int precision);`~~

```
__to_chars_result to_chars(char* first, char* last, *see below* value,
                           chars_format fmt, int precision);
```

13. *Requires*: `fmt` has the value of one of the enumerators of `chars_format`.
14. *Effects*: value is converted to a string in the style of `printf` in the `"C"` locale with the given precision.
15. *Throws*: Nothing.
16. *Remarks*: The implementation shall provide overloads for all floating-point types as the type of parameter `value`.

Modify Primitive numeric input conversions [**charconv.from.chars**]:

[...]

```
~~from_chars_result from_chars(const char* first, const char* last, floa~~
~~                             chars_format fmt = chars_format::general)~~
~~from_chars_result from_chars(const char* first, const char* last, doub~~
~~                             chars_format fmt = chars_format::general)~~
~~from_chars_result from_chars(const char* first, const char* last, long~~
~~                             chars_format fmt = chars_format::general)~~
```

```
from_chars_result from_chars(const char* first, const char* last, *see
                             chars_format fmt = chars_format::general)
```

6. *Requires*: `fmt` has the value of one of the enumerators of `chars_format`.
7. *Effects*: The pattern is the expected form of the subject sequence in the `"C"` locale, as described for `strtod`, except that
   - (7.1) the sign `'+'` may only appear in the exponent part;
   - (7.2) if `fmt` has `chars_format::scientific` set but not `chars_format::fixed`, the otherwise optional exponent part shall appear;
   - (7.3) if `fmt` has `chars_format::fixed` set but not `chars_format::scientific`, the optional exponent part shall not appear; and
   - (7.4) if `fmt` is `chars_format::hex`, the prefix `"0x"` or `"0X"` is assumed. [*Example*: The string `0x123` is parsed to have the value `0` with remaining characters `x123`. — *end example*]

   In any case, the resulting value is one of at most two floating-point values closest to the value of the string matching the pattern.
8. *Throws*: Nothing.
9. *Remarks*: The implementation shall provide overloads for all floating-point types as the type of parameter `value`.

Modify Complex numbers [**complex.numbers**] paragraph 2:

> 2. The effect of instantiating the template `complex` for any type ~~other than `float`, `double`, or `long double`~~ that is not a floating-point type is unspecified. The specializations ~~specializations `complex<float>`, `complex<double>`, and `complex<long double>`~~ of `complex` for floating-point types are literal types.

Modify Header `<complex>` synopsis [**complex.syn**]:

> [...]
>
> ```
> ~~// [complex.special], specializations~~
> ~~template<> class complex<float>;~~
> ~~template<> class complex<double>;~~
> ~~template<> class complex<long double>;~~
> ```

Modify Class template `complex` [**complex**]:

```
namespace std {
  template<class T> class complex {
  public:
    using value_type = T;

    constexpr complex(const T& re = T(), const T& im = T());
```

```
    constexpr complex(const complex&);
    template<class X> constexpr complex(const complex<X>&);
```

```
    constexpr complex(const complex&) = default;
    template<class X> constexpr explicit(*see below*) complex(const comp
```

```
    constexpr T real() const;
    constexpr void real(T);
    constexpr T imag() const;
    constexpr void imag(T);

    constexpr complex& operator= (const T&);
    constexpr complex& operator+=(const T&);
    constexpr complex& operator-=(const T&);
    constexpr complex& operator*=(const T&);
    constexpr complex& operator/=(const T&);

    constexpr complex& operator=(const complex&);
    template<class X> constexpr complex& operator= (const complex<X>&);
    template<class X> constexpr complex& operator+=(const complex<X>&);
    template<class X> constexpr complex& operator-=(const complex<X>&);
    template<class X> constexpr complex& operator*=(const complex<X>&);
    template<class X> constexpr complex& operator/=(const complex<X>&);
  };
}
```

Remove Specializations [**complex.special**]:

```
namespace std {
  template<> class complex<float> {
  public:
    using value_type = float;

    constexpr complex(float re = 0.0f, float im = 0.0f);
    constexpr complex(const complex<float>&) = default;
    constexpr explicit complex(const complex<double>&);
    constexpr explicit complex(const complex<long double>&);

    constexpr float real() const;
    constexpr void real(float);
    constexpr float imag() const;
    constexpr void imag(float);

    constexpr complex& operator= (float);
    constexpr complex& operator+=(float);
    constexpr complex& operator-=(float);
    constexpr complex& operator*=(float);
    constexpr complex& operator/=(float);

    constexpr complex& operator=(const complex&);
    template<class X> constexpr complex& operator= (const complex<X>&);
    template<class X> constexpr complex& operator+=(const complex<X>&);
    template<class X> constexpr complex& operator-=(const complex<X>&);
    template<class X> constexpr complex& operator*=(const complex<X>&);
    template<class X> constexpr complex& operator/=(const complex<X>&);
  };

  template<> class complex<double> {
  public:
    using value_type = double;

    constexpr complex(double re = 0.0, double im = 0.0);
    constexpr complex(const complex<float>&);
    constexpr complex(const complex<double>&) = default;
    constexpr explicit complex(const complex<long double>&);

    constexpr double real() const;
    constexpr void real(double);
    constexpr double imag() const;
    constexpr void imag(double);
```

```
    constexpr complex& operator= (double);
    constexpr complex& operator+=(double);
    constexpr complex& operator-=(double);
    constexpr complex& operator*=(double);
    constexpr complex& operator/=(double);

    constexpr complex& operator=(const complex&);
    template<class X> constexpr complex& operator= (const complex<X>&);
    template<class X> constexpr complex& operator+=(const complex<X>&);
    template<class X> constexpr complex& operator-=(const complex<X>&);
    template<class X> constexpr complex& operator*=(const complex<X>&);
    template<class X> constexpr complex& operator/=(const complex<X>&);
  };

  template<> class complex<long double> {
  public:
    using value_type = long double;

    constexpr complex(long double re = 0.0L, long double im = 0.0L);
    constexpr complex(const complex<float>&);
    constexpr complex(const complex<double>&);
    constexpr complex(const complex<long double>&) = default;

    constexpr long double real() const;
    constexpr void real(long double);
    constexpr long double imag() const;
    constexpr void imag(long double);

    constexpr complex& operator= (long double);
    constexpr complex& operator+=(long double);
    constexpr complex& operator-=(long double);
    constexpr complex& operator*=(long double);
    constexpr complex& operator/=(long double);

    constexpr complex& operator=(const complex&);
    template<class X> constexpr complex& operator= (const complex<X>&);
    template<class X> constexpr complex& operator+=(const complex<X>&);
    template<class X> constexpr complex& operator-=(const complex<X>&);
    template<class X> constexpr complex& operator*=(const complex<X>&);
    template<class X> constexpr complex& operator/=(const complex<X>&);
  };
}
```

Modify Member functions [**complex.members**] by inserting the following after paragraph 2:

```
template<class X> constexpr explicit(*see below*) complex(const complex<
```

3. *Effects*: Constructs an object of class `complex`.
4. *Ensures*: `real() == other.real() && imag() == other.imag()`.
5. *Remarks*: The expression inside `explicit` evaluates to `false` if and only if the floating-point conversion rank of `T` is greater than the floating-point conversion rank of `X`.

Modify Additional overloads [**cmplx.over**] paragraph 2 and 3:

2. The additional overloads shall be sufficient to ensure:

   - ~~(2.1) If the argument has type `long double`, then it is effectively cast to `complex<long double>`.~~
   - ~~(2.2) Otherwise, if the argument has type `double` or an integer type, then it is effectively cast to `complex<double>`.~~
   - ~~(2.3) Otherwise, if the argument has type `float`, then it is effectively cast to `complex<float>`.~~
   - (2.1) If the argument has a floating-point type `T`, then it is effectively cast to `complex<T>`.
   - (2.2) Otherwise, if the argument has an integer type, then it is effectively cast to `complex<double>`.

3. Function template `pow` shall have additional overloads sufficient to ensure, for a call with at least one argument of type `complex<T>`:

   - ~~(3.1) If either argument has type `complex<long double>` or type `long double`, then both arguments are effectively cast to `complex<long double>`.~~
   - ~~(3.2) Otherwise, if either argument has type `complex<double>`, `double`, or an integer type, then both arguments are effectively cast to `complex<double>`.~~
   - ~~(3.3) Otherwise, if either argument has type `complex<float>` or `float`, then both arguments are effectively cast to `complex<float>`.~~
   - (3.1) If the type of one of the arguments is `complex<T1>` and the type of the other is `complex<T2>`, then both arguments are effectively cast to `complex<TR>`, where `TR` is `T1` if `T1` has a higher floating-point conversion rank than `T2`, otherwise `T2`. If the floating-point conversion ranks of `T1` and `T2` are not ordered, the program is ill-formed.
   - (3.2) Otherwise, if the type of one of the arguments is `complex<T1>` and the type of the other is a floating-point type `T2`, then both arguments are effectively cast to `complex<TR>`, where `TR` is `T1` if `T1` has a higher floating-point conversion rank than `T2`, otherwise `T2`.
   - (3.3) Otherwise, both arguments are effectively cast to `complex<T>`.

Modify Header `<cmath>` synopsis [**cmath.syn**] paragraph 2 and add paragraph 3:

2. For each set of overloaded functions within `<cmath>`, with the exception of `abs`, there shall be additional overloads sufficient to ensure:

    1. ~~If any argument of arithmetic type corresponding to a double parameter has type long double, then all arguments of arithmetic type corresponding to double parameters are effectively cast to long double.~~

    2. ~~Otherwise, if any argument of arithmetic type corresponding to a double parameter has type double or an integer type, then all arguments of arithmetic type corresponding to double parameters are effectively cast to double.~~

    3. If all arguments of arithmetic types corresponding to `double` parameters have floating-point types, then all arguments of arithmetic type corresponding to `double` parameters have type that is the type among the argument types with the highest floating-point conversion rank. If that type is an extended floating-point type, then the return type is also that type. If any two types `T1` and `T2` among the arithmetic type arguments have floating-point conversion ranks that are not ordered, the program is ill-formed.

    4. Otherwise, if any argument of arithmetic type corresponding to a `double` parameter has a floating-point type, then all arguments of arithmetic type corresponding to `double` parameters are effectively cast to that of parameters of floating-point type that is the type with the highest floating-point conversion rank among those of argument types that are floating-point.

    5. Otherwise, all arguments of arithmetic type corresponding to `double` parameters have type `float`.

3. There shall be additional overloads of `abs` for each extended floating-point type `T`. Those overloads shall have the signature `T abs(T j)` and return the absolute value of `j`.

Note: LWG question: should the signatures be somehow added to the synopsis itself?

Note: We have tried to capture what the current specification says, without having to add three identical items into this wording to cover, respectively, EFPTs bigger than `long double`, EFPTs between `double` and `float`, and EFPTs smaller than `float`. We don't have anything against reverting to that, but wanted to try this more generic way of describing the behavior.

Note: We are pretty sure this new paragraph 3 is not the way to spell it, so we will welcome any suggestions.

§ References

§ Informative References

[P0192]

Michał Dominiak; et al. `short float` and fixed-size floating point types. URL: https://wg21.link/P0192

[P0645]

Victor Zverovich. Text Formatting. URL: https://wg21.link/P0645

[P1468]

Michał Dominiak; Boris Fomitchev; Sergei Nikolaev. Fixed-layout floating-point type aliasess. URL: https://wg21.link/P1468